MediConecta

TP INTEGRADOR · DESARROLLO DE APLICACIONES II

Documento Técnico
Entrega Obligatoria N.º 1

Arquitectura, componentes, patrones y decisiones de diseño

Materia	Desarrollo de Aplicaciones II (3.4.218), comisión Lunes TM, 2.º cuatrimestre 2026
Grupo	MediConecta Grupo 04: Agustín Bodas · Maxi Bona · Sergio Ariel Casanova
Dominio	MediConecta: turnos médicos presenciales, telemedicina y facturación a obras sociales
Alcance	`ServicioDeUsuarios` · `ServicioDeTurnos` · `ServicioDeHistoriaClinica`
Stack	Jakarta EE 10 (JSF 4 + PrimeFaces 14, EJB, CDI, JPA/Hibernate, Jakarta Security) sobre WildFly 41 + PostgreSQL
Fecha	14/09/2026 (Unidades I, II y III)

1. Introducción y alcance

MediConecta conecta pacientes con profesionales de salud independientes y pequeñas clínicas. La consigna plantea un sistema de ocho componentes de negocio con integraciones síncronas (SOAP/REST) y asincrónicas (mensajería). En esta entrega se implementaron y desplegaron **tres componentes completos** que, juntos, resuelven un caso de uso de punta a punta sin depender de sistemas externos: un paciente se registra, reserva y confirma un turno, y el profesional documenta la atención en la historia clínica.

Las integraciones con obras sociales (SOAP), pagos y videoconsulta (REST) y la mensajería de notificaciones corresponden a las Unidades IV a VI. Se incorporan en la Entrega Parcial N.º 2 (19/10) y en la Entrega Obligatoria N.º 2 (09/11).

1.1 Requisitos de la entrega y dónde se cumplen

Requisito de la consigna	Evidencia en el sistema	
Al menos 3 componentes implementados y desplegados, cada uno en capas	3 EJB con vistas JSF, beans de negocio y DAOs JPA separados en paquetes `web` / `negocio` / `dao` (sección 3)	✓
Al menos 1 componente stateful y 1 stateless, con ciclo de vida gestionado por el contenedor	`ServicioDeTurnosBean` `@Stateful` con `@StatefulTimeout`, `@PostConstruct`, `@PrePassivate`, `@PostActivate` y `@PreDestroy`. Los otros dos son `@Stateless` (sección 4)	✓
Al menos 3 patrones de diseño aplicados y justificados	DAO, Facade y Factory (sección 5)	✓
Seguridad declarativa: autenticación y autorización por rol en al menos una operación sensible	`@DatabaseIdentityStoreDefinition` + PBKDF2 y `@RolesAllowed` en 10 operaciones de negocio, entre ellas la carga de diagnósticos y recetas (sección 6)	✓
Documento técnico de 5 a 8 páginas	Este documento, con capturas de la aplicación en funcionamiento (sección 7)	✓

2. Arquitectura general

El sistema usa una **arquitectura en tres capas**, replicada en cada componente y desplegada como un único WAR en WildFly. La **presentación** (vistas Facelets + managed beans CDI) solo captura y valida el formato de entrada, y delega en la capa de negocio a través de la **interfaz local** del EJB (`@Local`). El **negocio** (EJB) aplica las reglas del dominio y queda gobernado por el contenedor, que maneja transacciones JTA, seguridad y ciclo de vida. Los **datos** se aíslan detrás de DAOs que encapsulan el `EntityManager` de JPA sobre el datasource JTA `java:/mediconectaDS`.

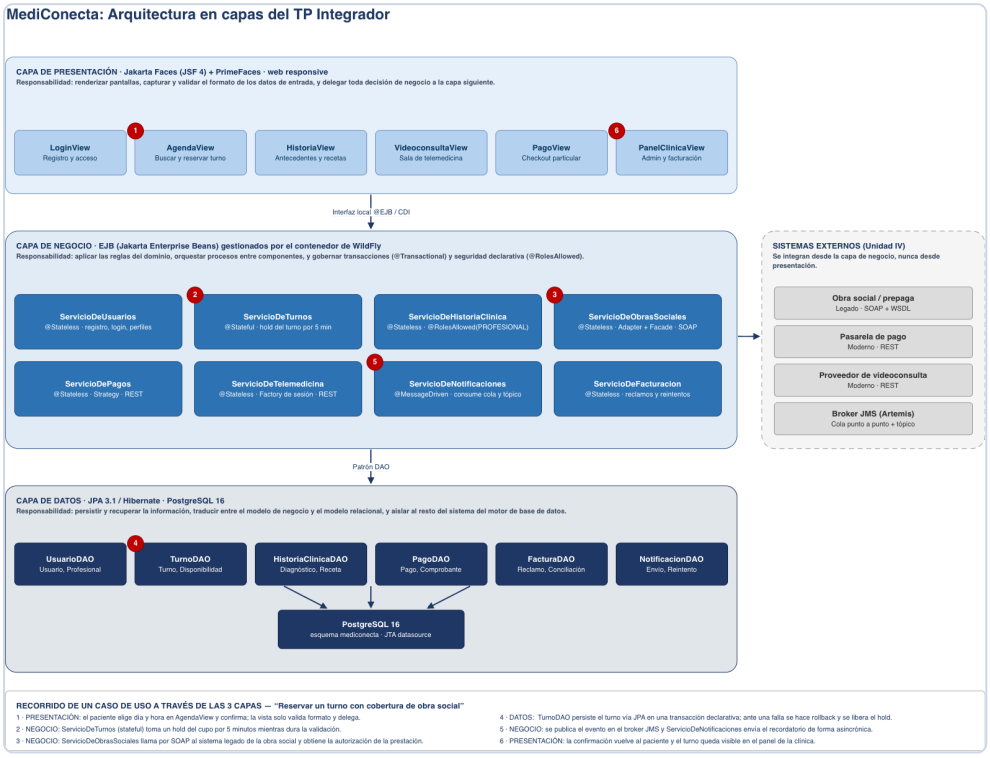


Fig. 1. Arquitectura en capas planificada para el sistema completo, con los sistemas externos que se integran desde la capa de negocio en las Unidades IV a VI.

2.1 Componentes identificados y estado

Componente	Responsabilidad	Tipo / integración	Estado
`ServicioDeUsuarios`	Registro y autenticación de pacientes y profesionales	@Stateless	Implementado
`ServicioDeTurnos`	Disponibilidad, hold de 5 min, confirmación y cancelación	@Stateful	Implementado
`ServicioDeHistoriaClinica`	Diagnósticos y recetas del paciente	@Stateless + @RolesAllowed	Implementado
`ServicioDeObrasSociales`	Validar cobertura y autorizar prestaciones	Adapter · SOAP (legado)	Entrega 2
`ServicioDePagos`	Cobro de turnos particulares	REST (pasarela)	Entrega 2
`ServicioDeTelemedicina`	Sesiones de videoconsulta	REST (proveedor)	Entrega 2
`ServicioDeNotificaciones`	Recordatorios y alertas multicanal	MDB · cola / tópico	Entrega 2
`ServicioDeFacturacion`	Reclamos a obras sociales y reintentos	@Stateless · asincrónico	Final

Por qué estos tres primero: son los únicos que forman un flujo de negocio completo y verificable en vivo sin sistemas externos. Además cubren los tres tipos de decisión que pide esta unidad: un componente stateful, componentes stateless y una operación con autorización por rol.

3. Componentes implementados

3.1 ServicioDeUsuarios (`@Stateless`)

Responsabilidad	Alta de pacientes y profesionales con contraseña hasheada, búsqueda por email y listado de profesionales.
Interfaz local	`registrar(Rol, DatosRegistro, passwordPlano)` · `buscarPorEmail(email)` · `listarProfesionales()`
Capas	`login.xhtml` + `LoginBean` → `ServicioDeUsuariosBean` (+ `UsuarioFactory`) → `UsuarioDAO`
Modelo	`Usuario` (abstracta) → `Paciente` (obra social) / `Profesional` (matrícula, especialidad), herencia `SINGLE_TABLE`
Reglas	Email único: lanza `EmailYaRegistradoException` (`@ApplicationException(rollback = true)`). Hash PBKDF2 con `Pbkdf2PasswordHash`, inicializado en `@PostConstruct`.

3.2 ServicioDeTurnos (`@Stateful`)

Responsabilidad	Publicación de disponibilidad, búsqueda, reserva temporal (hold), confirmación y cancelación de turnos.
Interfaz local	`publicarDisponibilidad` · `buscarDisponibilidad` · `reservarHold` · `confirmar` · `getTurnoEnHold` · `cancelar` · `buscarConfirmadosDePaciente` · `buscarConfirmadosDeProfesional`
Capas	`agenda.xhtml` + `AgendaBean` (`@SessionScoped`) → `ServicioDeTurnosBean` → `TurnoDAO`
Modelo	`Turno` con máquina de estados `DISPONIBLE → HOLD → CONFIRMADO` (vuelve a `DISPONIBLE` al expirar o cancelar) y `@Version` para bloqueo optimista.
Estado conversacional	`turnoEnHoldId`: el turno que el paciente tiene retenido en esta conversación.

3.3 ServicioDeHistoriaClinica (`@Stateless`)

Responsabilidad	Consulta y alta de entradas clínicas (diagnósticos y recetas). Son datos de alta sensibilidad.
Interfaz local	`obtenerDePaciente(pacienteId)` · `agregarEntrada(pacienteId, profesionalId, turnoId, tipo, descripcion)`
Capas	`historia.xhtml` + `HistoriaBean` (`@ViewScoped`) → `ServicioDeHistoriaClinicaBean` → `HistoriaClinicaDAO`
Modelo	`HistoriaClinica` (1 a 1 con `Paciente`) → N `EntradaClinica` (`DIAGNOSTICO` / `RECETA`, profesional, turno asociado y fecha)
Seguridad	Lectura: `@RolesAllowed({"PACIENTE","PROFESIONAL"})`. Escritura: `@RolesAllowed("PROFESIONAL")`.

> **Transacciones gestionadas por el contenedor**
> Todos los métodos de negocio corren con la transacción JTA por defecto de los EJB (`REQUIRED`). Las excepciones de dominio se anotan con `@ApplicationException(rollback = true)` para que hagan rollback sin ser tratadas como fallas de sistema. Sin esa anotación, al escaparse de un bean `@Stateful` el contenedor las envuelve en `EJBException` y **destruye la instancia**, y el paciente pierde su conversación de reserva.

4. Componente stateful vs. stateless

ServicioDeTurnos es stateful porque reservar un turno es una *conversación* con un paciente concreto. Cuando el paciente elige un horario, ese cupo queda retenido (HOLD) durante un tiempo acotado mientras decide confirmar, y ningún otro paciente puede tomarlo. El bean recuerda qué turno retiene cada paciente (`turnoEnHoldId`) entre la invocación `reservarHold` y la invocación `confirmar`. Ese estado atado a un cliente es justamente lo que modela un EJB `@Stateful`.

```
@Stateful
@StatefulTimeout(value = 5, unit = TimeUnit.MINUTES)
public class ServicioDeTurnosBean implements ServicioDeTurnos {
    private Long turnoEnHoldId;                      // estado conversacional

    @PostConstruct void iniciarConversacion() { ... }   // el contenedor crea la conversación
    @PrePassivate  void antesDePasivar()      { ... }   // puede serializarla a disco
    @PostActivate  void despuesDeActivar()    { ... }

    @PreDestroy                                         // al vencer el timeout sin confirmar
    void liberarHoldSiQuedoPendiente() {
        if (turnoEnHoldId != null) { /* turno HOLD → DISPONIBLE */ }
    }
}
```

> **Evidencia de ciclo de vida gestionado por el contenedor**
>
> No existe ninguna tarea programada propia que expire los holds. Si el paciente abandona la reserva, el **contenedor EJB** destruye la instancia al cumplirse el `@StatefulTimeout` de 5 minutos e invoca `@PreDestroy`, que devuelve el turno a `DISPONIBLE` y deja en el log de WildFly el mensaje "*Hold del turno N expiró sin confirmación; el contenedor liberó el turno automáticamente*". `@PrePassivate` y `@PostActivate` registran cuándo WildFly pasiva o reactiva la conversación.

ServicioDeUsuarios y ServicioDeHistoriaClinica son stateless porque cada operación es atómica y autocontenida. Registrar un usuario, buscarlo, consultar una historia o agregarle una entrada no requiere recordar nada entre invocaciones: toda la información llega por parámetro o está en la base. Así el contenedor puede reutilizar un *pool* de instancias entre todos los requests. Hacerlos stateful obligaría a mantener una instancia por cliente sin ningún beneficio. `ServicioDeUsuariosBean` también usa `@PostConstruct` para inicializar el algoritmo de hash una sola vez por instancia del pool.

Aspecto	ServicioDeTurnos (@Stateful)	Usuarios / HistoriaClinica (@Stateless)
Instancias	Una por cliente (sesión del paciente)	Pool compartido entre todos los requests
Estado entre llamadas	Sí: turno en hold	No
Fin de vida	Timeout de 5 min → `@PreDestroy` libera el turno	Lo decide el contenedor (tamaño del pool)
Callbacks usados	`@PostConstruct`, `@PrePassivate`, `@PostActivate`, `@PreDestroy`	`@PostConstruct` (Usuarios)

El hold y el bloqueo optimista se complementan. El hold evita que dos pacientes vean el mismo turno como disponible durante la decisión. `@Version` en `Turno` garantiza que, si dos reservas llegan en el mismo instante, solo una se persiste y la otra falla con conflicto de versión.

5. Patrones de diseño aplicados

5.1 DAO (Data Access Object)

Problema: la lógica de negocio no debe depender de cómo se persisten las entidades. **Aplicación:** `UsuarioDAO`, `TurnoDAO` e `HistoriaClinicaDAO` son beans CDI `@ApplicationScoped` que encapsulan el `EntityManager` y exponen operaciones del dominio (`existeEmail`, `buscarDisponibles`, `buscarConfirmadosDePaciente`). Los EJB nunca escriben JPQL. **Alternativa descartada:** inyectar el `EntityManager` directamente en los EJB. Mezclaría consultas con reglas de negocio, las mismas consultas se repetirían en varios servicios y la lógica sería difícil de probar de forma aislada.

5.2 Facade

Problema: reservar un turno no es una operación única. Es un subproceso con estados, validaciones de concurrencia y expiración. **Aplicación:** `ServicioDeTurnosBean` es la fachada de ese subproceso. La vista solo invoca métodos simples (`buscarDisponibilidad`, `reservarHold`, `confirmar`, `cancelar`) y no conoce la máquina de estados de `Turno`, el bloqueo optimista, el timeout del contenedor ni los DAOs involucrados (`TurnoDAO` y `UsuarioDAO`). **Alternativa descartada:** que `AgendaBean` orquestara los DAOs. Dispersaría reglas de negocio en la presentación y dejaría esos pasos fuera del control transaccional y de seguridad del contenedor.

5.3 Factory

Problema: el usuario a crear varía según el rol elegido en el registro, y cada subtipo tiene datos propios. **Aplicación:** `UsuarioFactory.crear(Rol, DatosRegistro, passwordHasheado)` decide qué subclase concreta instanciar. Un `switch` sobre el enum devuelve un `Paciente` o un `Profesional`, y `ServicioDeUsuariosBean` trabaja solo con la abstracción `Usuario`.

Alternativa descartada: un `if/else` dentro del EJB. Cada nuevo tipo de usuario, como el *Administrador de clínica* previsto por la consigna, obligaría a modificar el componente de negocio. Con la fábrica, el cambio queda en un único punto.

6. Seguridad declarativa

Autenticación. `ConfiguracionSeguridad` declara un `@DatabaseIdentityStoreDefinition` de Jakarta Security. Valida las credenciales contra la tabla `usuario` (`callerQuery` para el hash y `groupsQuery` para el rol) y compara las contraseñas con `Pbkdf2PasswordHash`: nunca se guardan en texto plano. El login usa un formulario propio que llama a `SecurityContext.authenticate()`. `MecanismoAutenticacion`, un `HttpAuthenticationMechanism`, delega la validación en ese identity store y notifica al contenedor el principal y su rol.

Autorización. Se aplica en la capa de negocio con `@RolesAllowed`, de modo que no depende de lo que muestre la vista:

Operación	Roles permitidos	Por qué
`HistoriaClinica.agregarEntrada`	PROFESIONAL	Solo un profesional puede escribir diagnósticos o recetas
`HistoriaClinica.obtenerDePaciente`	PACIENTE, PROFESIONAL	Datos sensibles: nunca accesibles sin autenticación
`Turnos.publicarDisponibilidad`	PROFESIONAL	Un paciente no puede crear horarios, aunque reenvíe un formulario
`Turnos.reservarHold / confirmar / cancelar`	PACIENTE	`cancelar` valida además que el turno sea del paciente

Si un usuario sin el rol invoca la operación, el contenedor lanza `EJBAccessException` antes de ejecutar el método. Como defensa adicional, `NoCacheFilter` evita que el botón "atrás" muestre páginas cacheadas de otro usuario, y `error.xhtml` reemplaza el stack trace por una pantalla de error genérica.

7. Demo: flujo de punta a punta

Las siguientes capturas corresponden a la aplicación desplegada en WildFly y recorren el flujo que se muestra en la defensa oral. Todos los datos se cargaron desde la propia interfaz.

1. **Registro y login** (`ServicioDeUsuarios`): la profesional y el paciente se registran (`UsuarioFactory`) e inician sesión.
2. **Publicación** (`ServicioDeTurnos`, rol PROFESIONAL): la profesional publica sus horarios, que quedan en estado DISPONIBLE.
3. **Reserva con hold** (bean stateful, rol PACIENTE): el paciente elige un horario y el turno pasa a HOLD por 5 minutos.
4. **Confirmación**: el turno pasa a CONFIRMADO y aparece en "Mis turnos", donde el paciente puede cancelarlo.
5. **Historia clínica** (`@RolesAllowed("PROFESIONAL")`): la profesional carga un diagnóstico y una receta; el paciente los consulta en solo lectura.

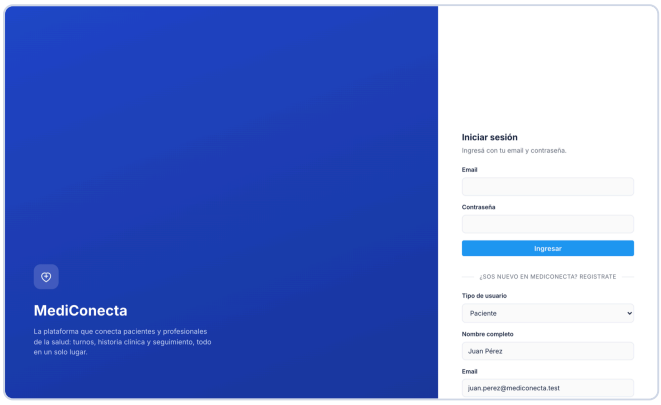


Fig. 2. Login y registro. Los campos cambian según el tipo de usuario (obra social o matrícula y especialidad).

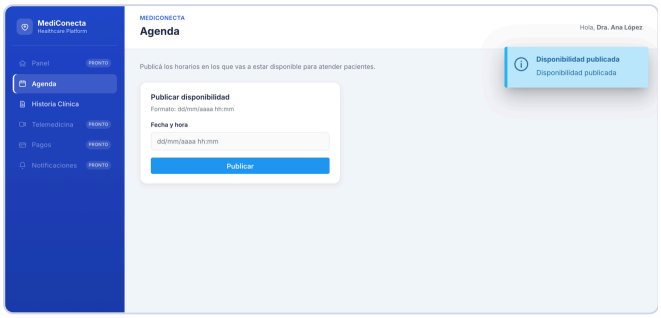


Fig. 3. Agenda de la profesional: publicación de disponibilidad.

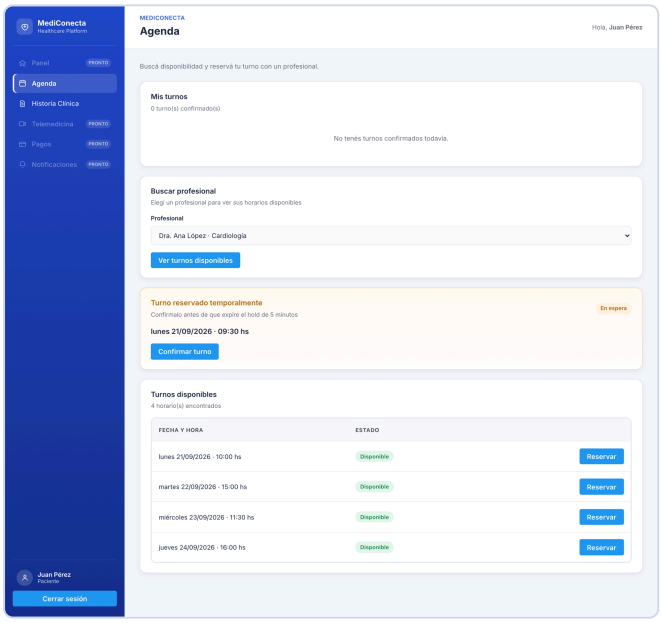


Fig. 4. Paciente con un turno en hold (bean `@Stateful`). El horario retenido ya no aparece en la lista de disponibles.

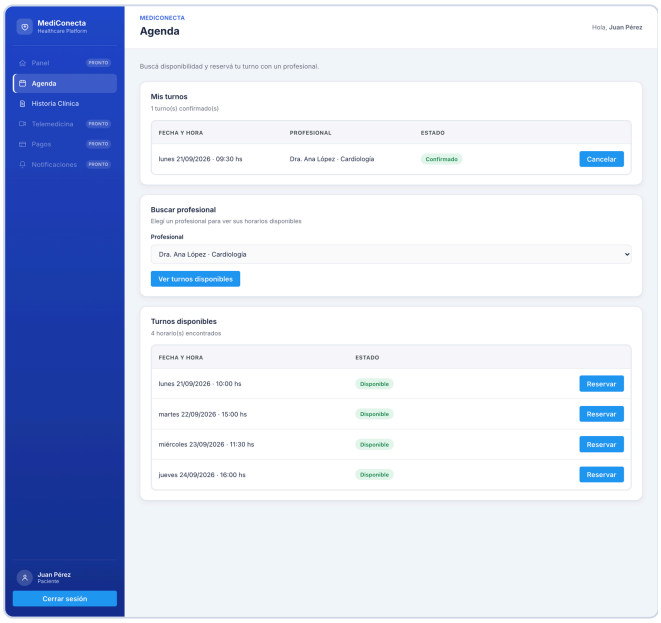


Fig. 5. Turno confirmado en "Mis turnos", con opción de cancelar.

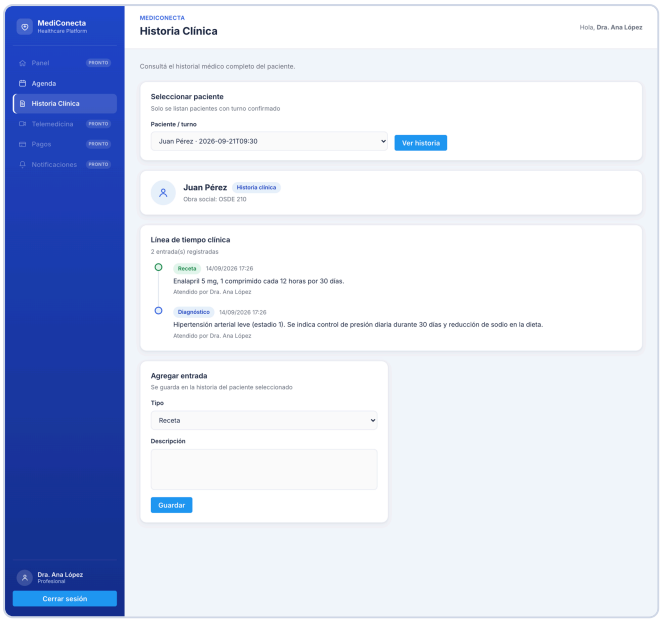


Fig. 6. La profesional selecciona un paciente con turno confirmado y carga un diagnóstico y una receta (operación protegida con `@RolesAllowed("PROFESIONAL")`).

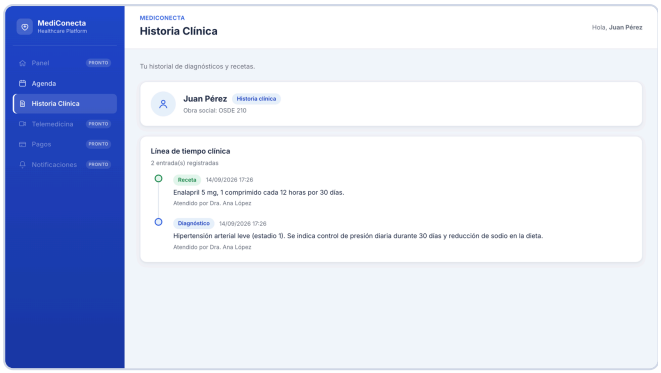


Fig. 7. El paciente consulta su historia clínica en solo lectura: el formulario de carga no se renderiza para su rol.

8. Stack tecnológico y decisiones de diseño

Decisión	Justificación
Jakarta EE 10 sobre WildFly 41	Es una de las opciones de stack de la cátedra y ofrece de forma nativa EJB stateful/stateless con ciclo de vida gestionado, transacciones JTA, JPA y Jakarta Security, sin frameworks de terceros para esas piezas.
JSF 4 + PrimeFaces 14	Presentación basada en componentes con binding directo a beans CDI, que mantiene la capa de presentación delgada. Los scopes se eligen según la vida del dato: `@SessionScoped` para la agenda, que acompaña la conversación de reserva, y `@ViewScoped` para la historia clínica.
Interfaces `@Local`	La presentación depende del contrato y no de la implementación. Para exponer un componente de forma remota o como servicio en entregas futuras no hay que tocar las vistas.
PostgreSQL gestionado (Supabase)	Base compartida por los tres integrantes sin instalar Postgres local. Se usa el *Session pooler* (IPv4) porque la conexión directa es IPv6. Las tablas se generan con `hbm2ddl=update`.
Herencia `SINGLE_TABLE`	Paciente y Profesional comparten identidad, credenciales y rol. Con una sola tabla, las consultas del identity store se resuelven sin JOINs.
`HttpAuthenticationMechanism` propio	`@FormAuthenticationMechanismDefinition` se comportaba de forma inconsistente en WildFly 41 con un login a medida. El mecanismo propio conserva el identity store declarativo. Además, hubo que desactivar `integrated-jaspi` en Elytron para que acepte principals que solo existen en la base.
`@Version` en Turno	Bloqueo optimista ante reservas simultáneas, complementario al hold temporal.

9. Seguimiento del equipo (Trello)

El avance se registra en el tablero **MediConecta Grupo 04** (`trello.com/b/7c84ZN3J`), con acceso de lectura para el docente. Tiene cuatro columnas: *Por hacer*, *En progreso*, *En revisión* y *Terminado*. Al momento de esta entrega hay 24 tarjetas: 11 terminadas, 8 en revisión, 4 en progreso y 1 pendiente, que corresponde a la planificación de la Entrega 2.

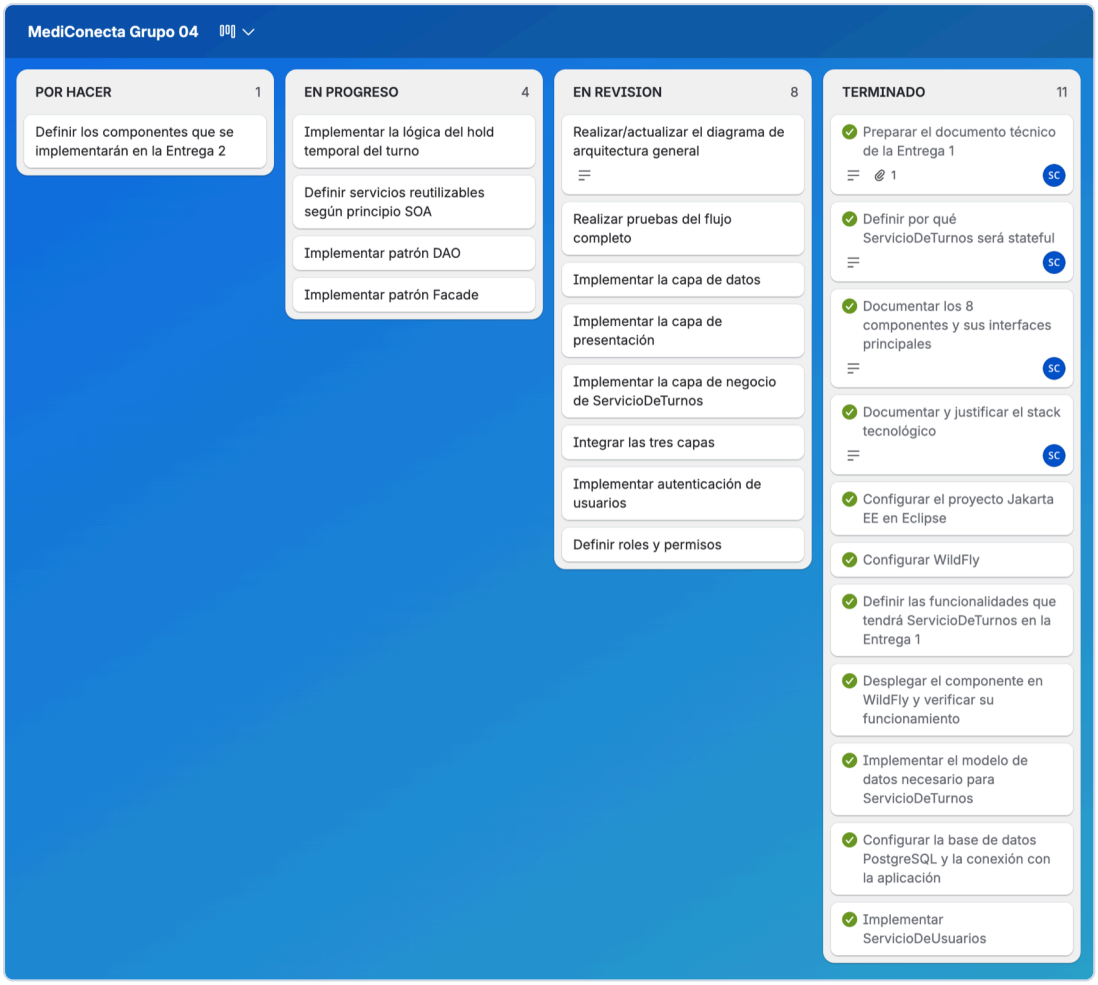


Fig. 8. Tablero Trello del grupo al 14/09/2026.

10. Uso de inteligencia artificial generativa

Según la sección 9 del TP Integrador, se declara el uso de IA generativa (Claude, de Anthropic) como asistente de productividad en esta entrega para:

- generar código repetitivo (boilerplate): entidades JPA, DAOs, EJB y vistas JSF/PrimeFaces;
- configurar el entorno local (JDK, Maven, WildFly) y su conexión con la base de datos;
- diagnosticar errores de integración entre Jakarta Security y WildFly 41 y revisar el manejo de excepciones en el bean stateful;
- redactar este documento y automatizar las capturas de pantalla.

Cada integrante puede explicar y defender individualmente cualquier parte del código o del diseño en la instancia oral, incluidas las decisiones tomadas con asistencia de IA.